

Node.js and Express.js Fundamentals

Course Information

Course Name: Node.js and Express.js Fundamentals

Level: Beginner to Intermediate

Prerequisites: Basic HTML, CSS, JavaScript, and React.js

Estimated Duration: 12–15 Hours

Course Description

Node.js is a JavaScript runtime that allows developers to execute JavaScript outside the browser. Express.js is a lightweight web framework built on Node.js that makes it easier to create web servers and APIs.

In this course, students will learn how to build backend applications using Node.js and Express.js. Topics include Node.js fundamentals, modules, npm, HTTP servers, Express routing, middleware, REST APIs, request and response handling, error handling, environment variables, and connecting a backend application with a frontend.

By the end of the course, students will be able to create a basic backend server and develop RESTful APIs for web applications.

Learning Outcomes

After completing this course, students will be able to:

- Explain what Node.js is.
- Understand how Node.js executes JavaScript outside the browser.
- Understand the Node.js project structure.
- Use npm to manage packages.
- Create and use Node.js modules.
- Create a basic HTTP server.
- Explain what Express.js is.
- Create an Express application.
- Define routes.
- Handle HTTP requests and responses.
- Use Express middleware.
- Work with JSON data.
- Build REST APIs.

- Handle API errors.
 - Use environment variables.
 - Connect a frontend application to a backend API.
 - Build a basic backend project.
-

Lesson 1: Introduction to Node.js

Learning Objectives

After this lesson, students will be able to:

- Define Node.js.
- Explain why Node.js is used.
- Understand JavaScript runtime environments.
- Identify differences between browser JavaScript and Node.js.

What is Node.js?

Node.js is a JavaScript runtime environment that allows JavaScript code to run outside a web browser.

Normally, JavaScript can run inside browsers such as:

- Chrome
- Firefox
- Edge

Node.js allows developers to use JavaScript for backend applications.

Why Use Node.js?

Node.js can be used to create:

- Web servers
 - REST APIs
 - Backend applications
 - Real-time applications
 - Command-line tools
 - Web services
-

Node.js Example

Create a file named:

```
app.js
```

Add:

```
console.log("Hello from Node.js");
```

Run:

```
node app.js
```

Output:

```
Hello from Node.js
```

Lesson 2: Node.js Project Setup

Creating a Project

Create a folder:

```
my-backend/
```

Open the folder in a terminal and initialize a Node.js project:

```
npm init
```

A `package.json` file will be created.

package.json

The file stores information about the project.

Example:

```
{  
  "name": "my-backend",  
  "version": "1.0.0",  
  "description": "My Node.js backend",  
  "main": "app.js"  
}
```

Project Structure

A basic project may look like:

```
my-backend/  
|  
├─ node_modules/  
├─ app.js  
├─ package.json  
└─ package-lock.json
```

Lesson 3: Modules

What are Modules?

Modules allow developers to organize code into separate files.

Node.js provides built-in modules as well as support for external modules.

Examples include:

- fs
- http
- path
- os

Importing a Module

Example:

```
const os = require("os");

console.log(os.platform());
```

File System Module

The `fs` module can be used to work with files.

Example:

```
const fs = require("fs");

fs.writeFileSync(
  "message.txt",
  "Hello Node.js"
);
```

This creates a file called:

```
message.txt
```

Path Module

```
const path = require("path");

const filePath = path.join(
  __dirname,
  "data",
  "students.json"
);

console.log(filePath);
```

Lesson 4: npm and Packages

What is npm?

npm is the package manager commonly used with Node.js.

It allows developers to:

- Install packages
- Remove packages
- Update packages
- Manage project dependencies

Installing a Package

Example:

```
npm install express
```

The package will be added to the project dependencies.

package.json Dependencies

After installation, the project may contain:

```
{
  "dependencies": {
    "express": "^5.0.0"
  }
}
```

node_modules

Installed packages are stored inside:

```
node_modules/
```

The folder can contain many files because packages may have their own dependencies.

Lesson 5: Creating an HTTP Server

HTTP Server

Node.js provides the `http` module for creating web servers.

Example:

```
const http = require("http");

const server = http.createServer(
  (request, response) => {

    response.writeHead(
      200,
      {
        "Content-Type":
          "text/plain"
      }
    );

    response.end(
      "Hello from the server"
    );
  }
);

server.listen(3000, () => {

  console.log(
    "Server running on port 3000"
  );
});
```

Starting the Server

Run:

```
node app.js
```

The server will listen on:

```
http://localhost:3000
```

Request and Response

The server receives a request and sends a response.

```
Client
  |
  | HTTP Request
  ↓
Node.js Server
  |
  | HTTP Response
  ↓
Client
```

Lesson 6: Introduction to Express.js

What is Express.js?

Express.js is a web framework for Node.js.

It provides tools for creating:

- Web servers
- Routes
- APIs
- Middleware
- Backend applications

Installing Express

Run:

```
npm install express
```

Basic Express Application

```
const express = require("express");

const app = express();

app.listen(3000, () => {

  console.log(
    "Server running on port 3000"
  );

});
```

Sending a Response

```
app.get("/", (request, response) => {

  response.send(
    "Welcome to my backend"
  );

});
```

Lesson 7: Routing

What is Routing?

Routing determines how a server responds to requests made to different URLs.

Example:

```
app.get("/", (request, response) => {

  response.send("Home Page");

});
```

```
});  
  
app.get("/about", (request, response) => {  
  
    response.send("About Page");  
  
});
```

HTTP Methods

Common HTTP methods include:

- GET
- POST
- PUT
- PATCH
- DELETE

GET Request

Used to retrieve information.

```
app.get("/students", (request, response) => {  
  
    response.json([  
        {  
            id: 1,  
            name: "Ali"  
        }  
    ]);  
  
});
```

POST Request

Used to send or create information.

```
app.post(  
  "/students",  
  (request, response) => {  
    response.json({  
      message:  
        "Student created"  
    });  
  }  
);
```

Lesson 8: Request and Response

Request Object

The request object contains information sent by the client.

Example:

```
app.get(  
  "/students",  
  (request, response) => {  
    console.log(  
      request.method  
    );  
    console.log(  
      request.url  
    );  
    response.send(  
      "Students"  
    );  
  }  
);
```

Response Object

The response object is used to send data back to the client.

Example:

```
response.send(  
  "Hello Client"  
);
```

JSON Response

APIs commonly return JSON.

```
response.json({  
  name: "Ali",  
  age: 21,  
  course: "Computer Science"  
});
```

Lesson 9: Express Middleware

What is Middleware?

Middleware is a function that executes during the request-response process.

It can be used for:

- Logging
 - Authentication
 - Validation
 - Parsing request data
 - Error handling
-

Basic Middleware

```
app.use(
  (request, response, next) => {

    console.log(
      "Request received"
    );

    next();
  }
);
```

JSON Middleware

Express provides middleware for parsing JSON requests.

```
app.use(
  express.json()
);
```

This allows the server to read JSON request bodies.

Middleware Flow

```
Request
  ↓
Middleware
  ↓
Route
  ↓
Response
```

Lesson 10: Request Parameters and Query Strings

Route Parameters

Route parameters allow values to be included in the URL.

Example:

```
app.get(
  "/students/:id",
  (request, response) => {

    const id =
      request.params.id;

    response.json({
      studentId: id
    });
  }
);
```

Request:

/students/25

The value of `id` is:

25

Query Parameters

Query parameters are commonly used for filtering or searching.

Example:

/students?course=CS

Access them using:

```
app.get(
  "/students",
  (request, response) => {

    const course =
      request.query.course;

    response.json({
      course: course
    });
  }
);
```

Lesson 11: REST APIs

What is a REST API?

A REST API allows applications to communicate through HTTP requests.

A student API might contain:

```
GET    /students
GET    /students/:id
POST   /students
PUT    /students/:id
DELETE /students/:id
```

GET All Students

```
app.get(
  "/students",
  (request, response) => {

    response.json(students);

  }
);
```

GET One Student

```
app.get(
  "/students/:id",
  (request, response) => {

    const id =
      Number(request.params.id);

    const student =
      students.find(
        student =>
          student.id === id
      );

    response.json(student);
  }
);
```

POST Student

```
app.post(
  "/students",
  (request, response) => {

    const student =
      request.body;

    console.log(student);

    response.status(201).json({
      message:
        "Student created",
      student
    });
  }
);
```

DELETE Student

```
app.delete(
  "/students/:id",
  (request, response) => {

    const id =
      Number(request.params.id);

    response.json({
      message:
        `Student ${id} deleted`
    });
  }
);
```

Lesson 12: Status Codes

What are HTTP Status Codes?

HTTP status codes describe the result of a request.

Common codes include:

```
200 OK
201 Created
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
500 Internal Server Error
```

Sending a Status Code

```
response
  .status(201)
  .json({
    message:
```

```
        "Student created"
    });
```

Not Found Response

```
response
  .status(404)
  .json({
    message:
      "Student not found"
  });
```

Lesson 13: Error Handling

Why Error Handling is Important

Errors can occur because:

- Data is invalid.
- A resource does not exist.
- A server operation fails.
- An unexpected problem occurs.

Basic Error Handling

```
app.get(
  "/students/:id",
  (request, response) => {

    try {

      const id =
        Number(request.params.id);

      if (!id) {
        throw new Error(
          "Invalid student ID"
        );
      }
    }
```

```
        response.json({
            id: id
        });
    } catch (error) {

        response
            .status(400)
            .json({
                error:
                    error.message
            });
    }
}
);
```

Error Middleware

Express applications can also use centralized error-handling middleware.

```
app.use(
    (error, request, response, next) => {

        console.error(error);

        response
            .status(500)
            .json({
                message:
                    "Internal server error"
            });
    }
);
```

Lesson 14: Environment Variables

What are Environment Variables?

Environment variables store configuration values outside the source code.

Examples include:

- Port numbers
- Database URLs
- API keys
- Secret values

Example `.env`

```
PORT=3000
DATABASE_URL=mongodb://localhost:27017/app
```

Accessing Environment Variables

```
const port =
  process.env.PORT || 3000;
```

Why Use Environment Variables?

They help keep configuration separate from application code.

Sensitive information such as passwords and secret keys should not be directly placed into source code.

Lesson 15: Connecting React With Express

A React frontend can communicate with an Express backend through HTTP requests.

Architecture:

```
React Frontend
  |
  | HTTP Request
  ↓
Express API
  |
  ↓
```

Backend Logic

|

↓

Database

Express API

Backend:

```
app.get(
  "/api/students",
  (request, response) => {

    response.json([
      {
        id: 1,
        name: "Ali",
        course:
          "Web Development"
      },
      {
        id: 2,
        name: "Sara",
        course:
          "Computer Science"
      }
    ]);

  }
);
```

Fetching From React

React can request the API:

```
fetch(
  "http://localhost:3000/api/students"
)
  .then(response =>
    response.json()
  )
```

```
.then(data => {
  console.log(data);
});
```

Lesson 16: Complete REST API Example

The following example combines several concepts from the course.

```
const express = require("express");

const app = express();

app.use(express.json());

const students = [
  {
    id: 1,
    name: "Ali",
    course: "Web Development"
  },
  {
    id: 2,
    name: "Ahmed",
    course: "Computer Science"
  }
];

// Get all students
app.get(
  "/api/students",
  (request, response) => {

    response.json(students);

  }
);

// Get one student
app.get(
  "/api/students/:id",
  (request, response) => {

    const id =
      Number(request.params.id);
```

```

    const student =
      students.find(
        item =>
          item.id === id
      );

    if (!student) {

      return response
        .status(404)
        .json({
          message:
            "Student not found"
        });
    }

    response.json(student);
  }
);

// Create student
app.post(
  "/api/students",
  (request, response) => {

    const newStudent = {
      id: students.length + 1,
      name: request.body.name,
      course: request.body.course
    };

    students.push(newStudent);

    response
      .status(201)
      .json(newStudent);
  }
);

// Delete student
app.delete(
  "/api/students/:id",
  (request, response) => {

    const id =
      Number(request.params.id);

```

```
const index =
  students.findIndex(
    item =>
      item.id === id
  );

if (index === -1) {

  return response
    .status(404)
    .json({
      message:
        "Student not found"
    });
}

students.splice(index, 1);

response.json({
  message:
    "Student deleted"
});
}
);

app.listen(3000, () => {

  console.log(
    "Server running on port 3000"
  );

});
```

Mini Project

Student Management REST API

Create a backend application using Node.js and Express.js.

Requirements

The API should contain:

1. Student management

2. GET students endpoint
3. GET individual student endpoint
4. POST student endpoint
5. PUT student endpoint
6. DELETE student endpoint
7. JSON request and response handling
8. Error handling
9. HTTP status codes
10. Environment configuration

Student Data

Each student should contain:

```
ID
Name
Email
Age
Course
```

API Endpoints

```
GET    /api/students
GET    /api/students/:id
POST   /api/students
PUT    /api/students/:id
DELETE /api/students/:id
```

Additional Requirement

Create a simple React frontend that retrieves student data from the Express API.

The frontend should:

- Display students.
- Add students.
- Delete students.
- Show loading information.
- Display an error message when the API request fails.

Practice Exercises

Exercise 1

Create a Node.js application that prints:

```
Hello from my backend
```

Exercise 2

Create an Express server with the following routes:

```
/
/about
/contact
```

Return different messages from each route.

Exercise 3

Create a `/students` API endpoint that returns three students in JSON format.

Exercise 4

Create a route:

```
/students/:id
```

Return the student ID received from the URL.

Exercise 5

Create a POST endpoint that accepts:

- Name
- Email

- Course

Return the received information as JSON.

Exercise 6

Create a DELETE endpoint:

```
/students/:id
```

Return a confirmation message containing the student ID.

Exercise 7

Create middleware that logs:

- HTTP method
- URL
- Request time

for every incoming request.

Exercise 8

Create a React frontend that retrieves data from your Express API and displays the results in student cards.

Final Quiz

1. What is Node.js?
2. How is Node.js different from JavaScript running in a browser?
3. What is npm?
4. What is the purpose of `package.json`?
5. What is a Node.js module?
6. What is Express.js?
7. How do you create an Express application?
8. What is routing?
9. What is the difference between GET and POST?
10. What is the purpose of middleware?
11. What is the purpose of `express.json()`?
12. What are route parameters?

13. What are query parameters?
 14. What is a REST API?
 15. What are common HTTP methods?
 16. What is an HTTP status code?
 17. What does status code 404 mean?
 18. What does status code 500 mean?
 19. Why is error handling important?
 20. What are environment variables?
 21. Why should sensitive configuration values not be hard-coded?
 22. How can React communicate with an Express backend?
 23. What is the purpose of JSON in web APIs?
 24. What is the difference between frontend and backend development?
 25. What are the main components of a REST API?
-

Course Summary

After completing this course, students should understand the fundamentals of backend web development using Node.js and Express.js.

The major concepts covered are:

- Node.js
- Node.js project structure
- Modules
- npm
- Packages
- HTTP servers
- Express.js
- Routing
- HTTP methods
- Request and response objects
- Middleware
- Route parameters
- Query parameters
- REST APIs
- JSON
- HTTP status codes
- Error handling
- Environment variables
- React and Express communication
- Backend project organization
- REST API development

Next Step

After completing Node.js and Express.js Fundamentals, students can continue with **Database and Full-Stack Web Development**, where they can learn:

- MongoDB
- SQL databases
- Database CRUD operations
- Authentication
- JWT
- User registration and login
- Connecting databases with Express
- Full-stack React + Node.js applications
- Deployment

The recommended learning path is now:

HTML → CSS → JavaScript → React.js → Node.js & Express.js → Database & Full-Stack Development